

Obstacles for a Llama

O lamă vrea să călătorească prin Podișul Anzilor. Lama are o hartă a podișului sub forma unei grile de $N \times M$ celule pătrate. Rândurile hărții sunt numerotate de la 0 la $N - 1$ de sus în jos, iar coloanele sunt numerotate de la 0 la $M - 1$ de stânga la dreapta. Celula hărții din rândul i și coloana j ($0 \leq i < N, 0 \leq j < M$) este notată cu (i, j) .

Lama a studiat clima podișului și a descoperit că toate celulele din fiecare rând al hărții au aceeași **temperatură** și toate celulele din fiecare coloană a hărții au aceeași **umiditate**. Lama ți-a dat două tablouri de numere întregi T și H de lungime N și respectiv M . Aici $T[i]$ ($0 \leq i < N$) indică temperatura celulelor din rândul i , iar $H[j]$ ($0 \leq j < M$) indică umiditatea celulelor din coloana j .

Lama a studiat și flora platoului și a observat că o celulă (i, j) este **liberă de vegetație** dacă și numai dacă temperatura sa este mai mare decât umiditatea, formal $T[i] > H[j]$.

Lama poate călători peste platou doar urmând **căi valide**. O cale validă este o secvență de celule distincte care îndeplinesc următoarele condiții:

- Fiecare pereche de celule consecutive din cale are o latură comună.
- Toate celulele din cale sunt libere de vegetație.

Sarcina ta este să răspunzi la Q întrebări. Pentru fiecare întrebare, primești patru numere întregi: L, R, S și D . Trebuie să determini dacă există o cale validă astfel încât:

- Calea începe la celula $(0, S)$ și se termină la celula $(0, D)$.
- Toate celulele din cale se află în coloanele de L la R , inclusiv.

Este garantat că atât $(0, S)$ cât și $(0, D)$ sunt libere de vegetație.

Detalii de implementare

Prima funcție pe care trebuie să o implementezi este:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un tablou de lungime N care specifică temperatura pe fiecare rând.
- H : Un tablou de lungime M care specifică umiditatea din fiecare coloană.

- Această funcție este apelată exact o singură dată pentru fiecare test, înainte de orice apel către `can_reach`.

A doua funcție pe care trebuie să o implementezi este:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : numere întregi care descriu o întrebare.
- Această funcție se apelează de Q ori pentru fiecare test.

Această funcție ar trebui să returneze `true` dacă și numai dacă există o cale validă de la celula $(0, S)$ la celula $(0, D)$, astfel încât toate celulele din cale să se afle în coloanele de la L la R , inclusiv.

Restricții

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ pentru fiecare i astfel încât $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ pentru fiecare j astfel încât $0 \leq j < M$
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Ambele celule $(0, S)$ și $(0, D)$ sunt libere de vegetație.

Subtask-uri

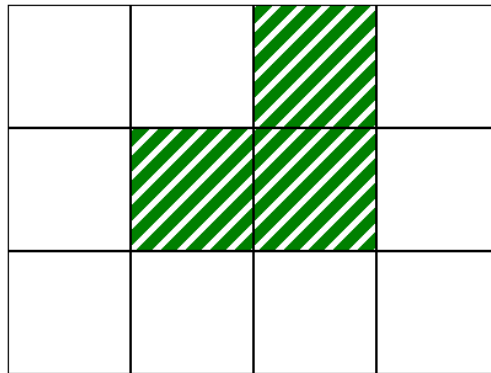
Subtask	Scor	Restricții suplimentare
1	10	$L = 0, R = M - 1$ pentru fiecare întrebare. $N = 1$.
2	14	$L = 0, R = M - 1$ pentru fiecare întrebare. $T[i - 1] \leq T[i]$ pentru fiecare i astfel încât $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ pentru fiecare întrebare. $N = 3$ și $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ pentru fiecare întrebare. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ pentru fiecare întrebare.
6	17	Fără restricții suplimentare.

Exemplu

Se consideră următorul apel:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

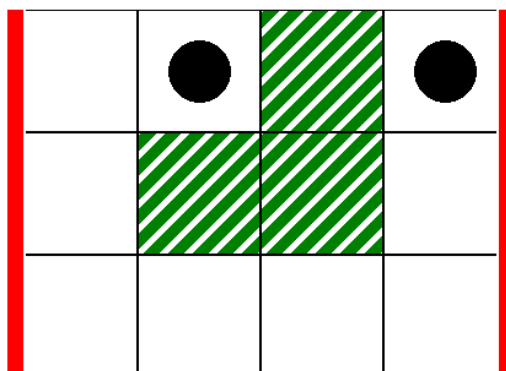
Acesta corespunde hărții din imaginea următoare, unde celulele albe sunt libere de vegetație:



Ca primă întrebare, se ia în considerare următorul apel:

```
can_reach(0, 3, 1, 3)
```

Acesta corespunde scenariului din imaginea următoare, unde liniile verticale groase indică intervalul de coloane de la $L = 0$ la $R = 3$, iar discurile negre indică celulele de început și de sfârșit:



În acest caz, lama poate ajunge de la celula (0, 1) la celula (0, 3) prin următoarea cale validă:

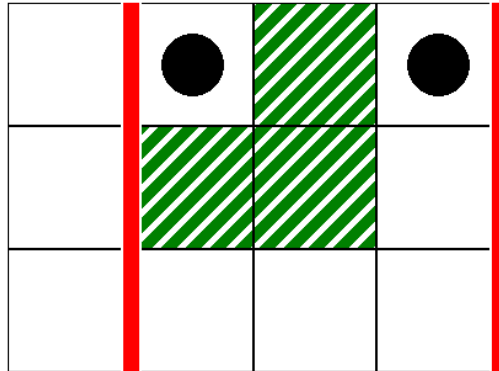
(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)

Prin urmare, acest apel ar trebui să returneze `true`.

Ca a doua întrebare, se ia în considerare următorul apel:

```
can_reach(1, 3, 1, 3)
```

Acesta corespunde scenariului din imaginea următoare:



În acest caz, nu există o cale validă de la celula $(0, 1)$ la celula $(0, 3)$, astfel încât toate celulele din cale să se afle în coloanele 1 până la 3, inclusiv. Prin urmare, acest apel ar trebui să returneze `false`.

Grader local

Format intrare:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aici, $L[k]$, $R[k]$, $S[k]$ și $D[k]$ ($0 \leq k < Q$) specifică parametrii pentru fiecare apel către `can_reach`.

Format ieșire:

```
A[0]
A[1]
...
A[Q-1]
```

Aici, $A[k]$ ($0 \leq k < Q$) este 1 dacă apelul `can_reach(L[k], R[k], S[k], D[k])` a returnat `true` și 0 în caz contrar.